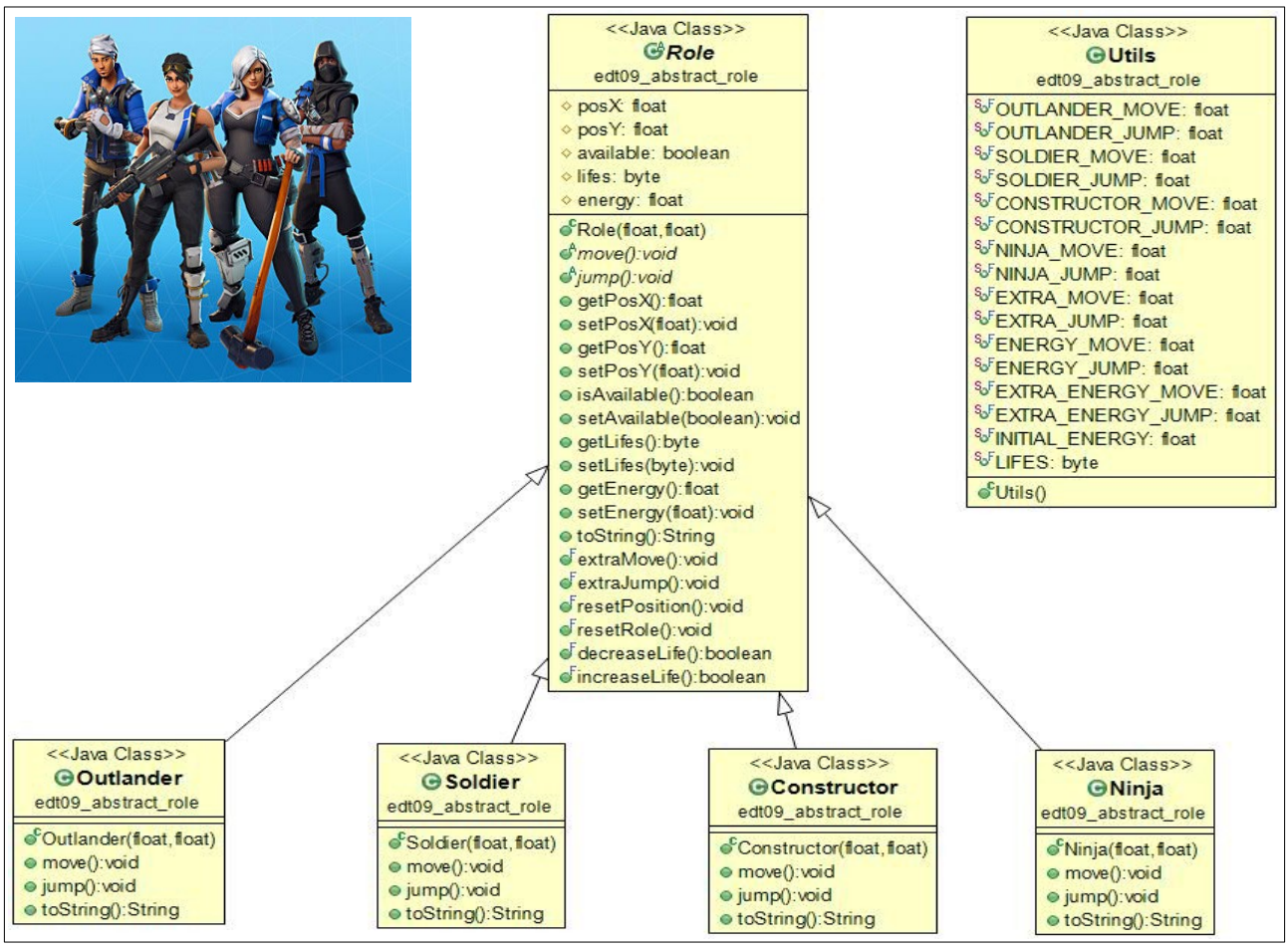
Role Abstract

a) Crea un directori de nom ***OOP***.
b) Crea un *package* amb el nom ***role*** ubicat dins del directori anterior.



Volem simular els comportament de 4 personatges d'un joc, els personatges són: **Outlander**, **Soldier**, **Constructor** i **Ninja**.

Els personatges tenen tots els mateixos atributs:

Attribute	Type	Default value
posX	float	
poxY	float	
available	boolean	true
lifes	byte	3
energy	float	100.0

Cada personatge es mourà i saltarà d'una manera diferent.

No volem que es pugui instanciar cap objecte de la classe **Role**, per tant s'haurà de definir com **abstracta**.

Copyright (C) Joan Sèculi (joan.seculi@gmail.com)

1. Mètodes abstractes

La classe **Role** conté els següents mètodes abstractes que les subclasses hauran de sobreescriure:

1.1 move() i jump()

El moviment està controlat per la posició X, i el salt està controlat per la posició Y. Els personatges només es poden moure o saltar si està disponible (atribut **available** està a *true*).

	Outlander	Soldier	Constructor	Ninja
move	8.5	10.5	5.0	8.0
jump	6.2	8.0	5.0	7.5

Cada vegada que es mou un personatge, la seva energia es redueix en 0.5, i cada vegada que salta, l'energia es redueix en 1. Només es podran moure si tenen suficient energia per fer-ho, en cas contrari no es mouran.

	Move	Jump
Energy	0.5	1.0

2. Mètodes final

2.1 extraMove() i extraJump()

Els mètodes **extraMove()** i **extraJump()** incrementa els valors X i Y de la següent manera.

	Desplaçament
extraMove	20.5
extraJump	25.2

Cada vegada que un personatge realitza un moviment extra, la seva energia es redueix en 2, i cada vegada que realitza un salt extra l'energia es redueix en 4. Només es podran moure si els personatges estan disponibles (atribut **available** està a *true*) i tenen suficient energia per fer-ho, en cas contrari no es mouran.

	Extra Move	Extra Jump
Energy	2	4

2.2 resetPosition() i resetRole()

resetPosition() situa el personatge a la posició x = 0 i y = 0.

resetRole() situa el personatge a la posició x = 0 i y = 0, canvia **available** a *true*, reinicialitza les vides a 3 i l'energia a 100.

2.3 decreaseLife() i decreaseLife()

decreaseLife() redueix el número de vides només si el personatge està disponible i al menys té 1 vida, retorna *true* si es pot realitzar l'acció.

increaseLife() incrementa el número de vides només si el personatge està disponible i té menys de 3 vides, retorna *true* si es pot realitzar l'acció.

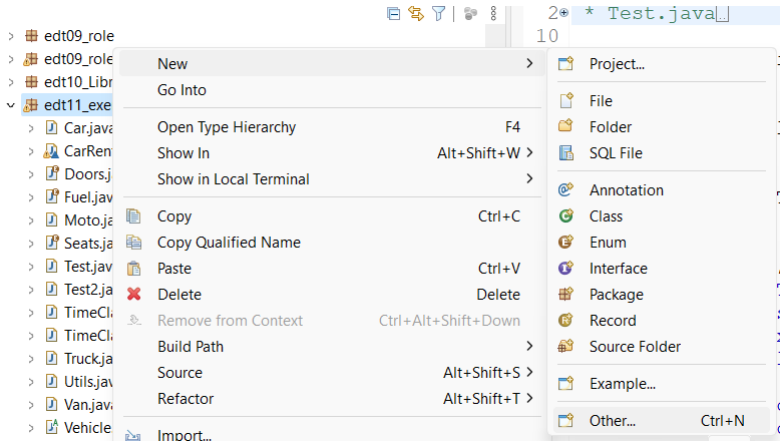
Copyright (C) Joan Sèculi (joan.seculi@gmail.com)

3. Test

Se't proporciona una classe de tests unitaris per realitzar tots els tests de les classes. S'ha d'instal·lar la llibreria de **JUnit** per poder executar aquesta classe. Segueix els passos següents:

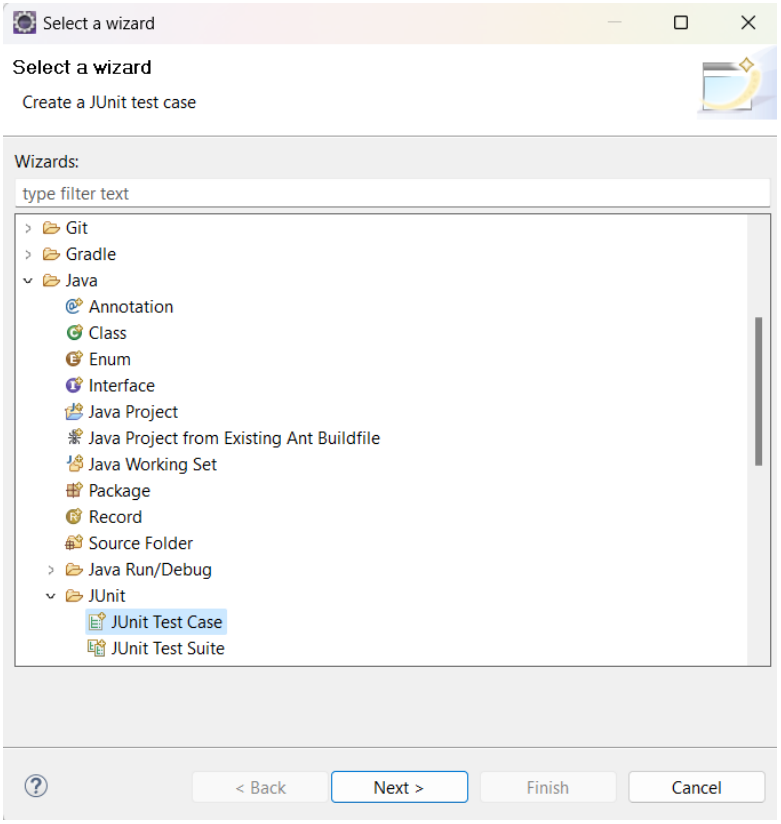
1. Afegir un nou

Selecciona el package de treball i amb el botó de la dreta del ratolí seleccionar en el menú:
New > Other ...



2. Selecciona la classe JUnit

En aquesta finestra, desplega "*Java*" i selecciona "*JUnit Test Case*".

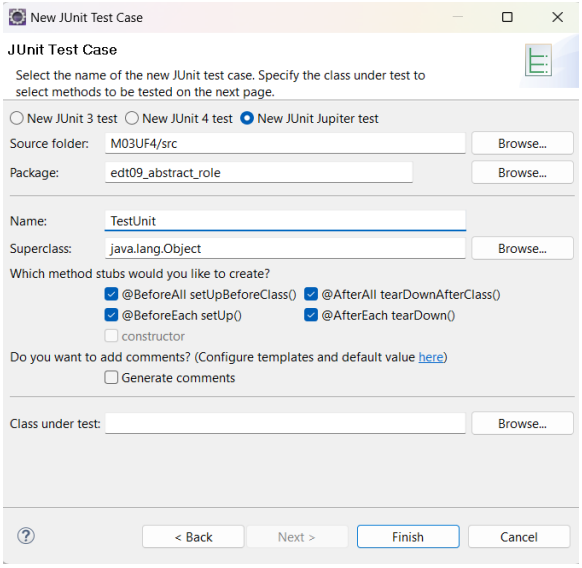


Fer clic a "*Next*"

Copyright (C) Joan Sèculi (joan.seculi@gmail.com)

3. Crear la classe JUnit

Seleccionar la versió "*New JUnit Jupiter test*" (última versió 5 de JUnit)
Donar un nom a la classe:
Name = CarRentUnitTests



Selecciona els mètodes *@BeforeAll, @AfterAll, @BeforeEach i @AfterEach* perquè aquests siguin inclosos a la classe.

Fer clic a "*Finish*".

4. Reemplaça la classe TestUnit.java

Un cop tinguis la classe de tests unitaris creada i les llibreries afegides en el projecte, podràs ara incloure la classe proporcionada en aquesta activitat, sobreescrius la classe **TestUnit.java** amb el fitxer o inclou tot el codi dins de la classe que acabes de crear.

Copyright (C) Joan Sèculi (joan.seculi@gmail.com)

Llicència

Copyright (C) Joan Sèculi (joan.seculi@gmail.com)
Permission is granted to copy, distribute and/or modify this document under the terms of the GNU Free Documentation License, Version 1.3 or any later version published by the Free Software Foundation; with no Invariant Sections, no Front-Cover Texts, and no Back-Cover Texts. A copy of the license is in http://www.gnu.org/licenses/fdl.txt